DOCUMENTATION TECHNIQUE Orchestration Automatique — Cluster Kubernetes

Documentation de Projet

Orchestration automatique des VM dans un cluster Kubernetes

Styve Landry SIME KOUNOU — 2026

---

0. FICHE PROJET

Projet	Orchestration Automatique des VM par Cluster Kubernetes
Technos	Proxmox · Kubernetes v1.30 · containerd · Gitea · Jenkins · Terraform · Ansible
Architecture	1 Master (SK-Master1) · 2 Workers (SK-Worker1 / SK-Worker2) · Namespace ci-cd
Objectif	Déployer un cluster Kubernetes, orchestrer les services CI/CD (Gitea + Jenkins) et automatiser le provisionnement d'infrastructure via Terraform
Résultat clé	Pipeline CI/CD fonctionnel : push Gitea → build Jenkins déclenché automatiquement (webhook)
Auteur	Styve Landry SIME KOUNOU
Date	Mars 2026

1. KUBERNETES CI/CD — INFRASTRUCTURE DE BASE

L'objectif est de concevoir et déployer une architecture complète combinant virtualisation (Proxmox), orchestration de conteneurs (Kubernetes) et outils CI/CD (Gitea + Jenkins).

1.1 Architecture Infrastructure — Préparation des VMs sur Proxmox

Trois machines virtuelles sont provisionnées sur Proxmox VE. Chaque nœud remplit un rôle distinct dans le cluster :

- Master (SK-Master1) — nœud de contrôle : minimum 2 vCPUs, 4 Go de RAM (etcd est exigeant en calcul)
- Worker 1 et Worker 2 — nœuds de calcul : mémoire suffisante pour Jenkins (JVM gourmande en RAM)
- Réseau — le Swap doit être désactivé sur les trois nœuds ; Kubernetes ne peut pas fonctionner avec la mémoire virtuelle activée

1.2 Installation du Cluster Kubernetes — Documentation Technique

Les étapes 1 à 4 s'exécutent sur les trois VMs. Les étapes suivantes sont spécifiques au Master puis aux Workers.

Étape 1 — Préparation du système (sur les 3 VMs)

Kubernetes exige un environnement Linux propre, sans interférence de la mémoire virtuelle.

```
# Passer en root
sudo su -

# Désactiver le swap immédiatement (requis par Kubernetes)
swapoff -a
# Désactiver le swap de manière permanente dans /etc/fstab
sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab

# Charger les modules noyau nécessaires
cat <<EOF | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
modprobe overlay
modprobe br_netfilter

# Configurer les paramètres sysctl pour le pont réseau
cat <<EOF | tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sysctl --system
```

Étape 2 — Installation du runtime containerd (sur les 3 VMs)

Kubernetes a besoin d'un moteur de conteneur (runtime). Containerd est le standard recommandé.

```
apt-get update && apt-get install -y ca-certificates curl gnupg

# Ajout du dépôt Docker (pour containerd)
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg \
  | gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/debian \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
  | tee /etc/apt/sources.list.d/docker.list > /dev/null

apt-get update && apt-get install -y containerd.io

# Configuration de containerd pour Kubernetes (cgroups systemd)
mkdir -p /etc/containerd
containerd config default | tee /etc/containerd/config.toml
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' \
  /etc/containerd/config.toml
systemctl restart containerd
```

Étape 3 — Installation des outils Kubernetes (sur les 3 VMs)

Installation de kubelet (agent), kubeadm (déploiement du cluster) et kubectl (interface de commande).

```
# Ajout du dépôt Kubernetes v1.30
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key \
  | gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
  https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' \
  | tee /etc/apt/sources.list.d/kubernetes.list

apt-get update
apt-get install -y kubelet kubeadm kubectl
apt-mark hold kubelet kubeadm kubectl
```

Étape 4 — Initialisation du Master (uniquement sur le Master)

La commande kubeadm init crée le « cerveau » du cluster : génération des certificats SSL internes et démarrage de l'API Server.

```
# Initialiser le cluster (remplacer <IP_MASTER> par l'IP réelle)
kubeadm init --pod-network-cidr=10.244.0.0/16 \
             --apiserver-advertise-address=<IP_MASTER>

# Configurer l'accès kubectl pour l'utilisateur courant
mkdir -p $HOME/.kube
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
chown $(id -u):$(id -g) $HOME/.kube/config

# Installer le plugin réseau Flannel (CNI)
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-
flannel.yml

# NOTE : copier la commande "kubeadm join ..." affichée en fin de sortie
# Elle sera utilisée pour connecter les Workers au cluster.
```

Étape 5 — Jonction des Workers (uniquement sur les 2 Workers)

```
# Coller ici la commande générée par le Master (exemple) :
kubeadm join <IP_MASTER>:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Étape 6 — Vérification du cluster (sur le Master)

```
# Les 3 nœuds doivent afficher le statut Ready
kubectl get nodes

# Exemple de sortie attendu :
# NAME       STATUS   ROLES           AGE   VERSION
# master     Ready    control-plane   16m   v1.30.14
# worker     Ready    <none>          9m    v1.30.14
# worker2    Ready    <none>          5m    v1.30.14
```

1.3 Namespace ci-cd — Isolation des services CI/CD

Plutôt que de déployer les outils dans le namespace par défaut, on crée un espace dédié ci-cd. L'isolation garantit que les erreurs de configuration n'affectent pas les autres services du cluster et permet une gestion fine des ressources CPU/RAM.

Méthode recommandée — déclaration YAML (Infrastructure as Code, réutilisable) :

```
# namespace-cicd.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: ci-cd
  labels:
    env: production
```

```
# Appliquer et vérifier
kubectl apply -f namespace-cicd.yaml
kubectl get ns ci-cd   # doit afficher 'Active'

# (Optionnel) Définir ci-cd comme namespace par défaut
kubectl config set-context --current --namespace=ci-cd
```

1.4 Déploiement de Gitea — Serveur Git dans Kubernetes

Trois objets Kubernetes sont créés pour Gitea :

- PersistentVolumeClaim (PVC) — stockage persistant pour les dépôts Git (survit aux redémarrages du conteneur)

- Deployment — décrit l'image à utiliser (gitea/gitea:latest), le nombre de réplicas et les volumes
- Service (NodePort) — expose l'interface web sur le réseau via un port fixe

```
# gitea-deploy.yaml
apiVersion: v1
kind: Service
metadata:
  name: gitea-service
  namespace: ci-cd
spec:
  selector:
    app: gitea
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30080   # Port accessible depuis le réseau
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gitea-v3
  namespace: ci-cd
spec:
  replicas: 1
  selector:
    matchLabels:
      app: gitea
  template:
    metadata:
      labels:
        app: gitea
    spec:
      containers:
        - name: gitea
          image: gitea/gitea:latest
          ports:
            - containerPort: 3000
```

```
kubectl apply -f gitea-deploy.yaml
kubectl get pods -n ci-cd
kubectl get svc -n ci-cd

# Validation : ouvrir http://<IP_WORKER>:30080 dans un navigateur
# → interface d'installation Gitea = déploiement réussi
```

1.5 Déploiement de Jenkins — Moteur d'Orchestration CI/CD

Jenkins nécessite un ServiceAccount Kubernetes pour pouvoir créer des agents (conteneurs) à la demande. La RAM allouée doit être suffisante (2 Gi minimum) pour que la JVM démarre correctement.

```yaml
# jenkins-deploy.yaml
apiVersion: v1
kind: Service
metadata:
  name: jenkins-service
  namespace: ci-cd
spec:
  selector:
    app: jenkins
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30000   # Interface Web Jenkins
    - port: 50000
      targetPort: 50000   # Communication agents/master
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jenkins
  namespace: ci-cd
spec:
  replicas: 1
  selector:
    matchLabels:
      app: jenkins
  template:
    metadata:
      labels:
        app: jenkins
    spec:
      containers:
        - name: jenkins
          image: jenkins/jenkins:lts
          resources:
            requests:
              memory: "2Gi"
            limits:
              memory: "2Gi"
          ports:
            - containerPort: 8080
            - containerPort: 50000
```

```
kubectl apply -f jenkins-deploy.yaml

# Récupérer le mot de passe initial (affiché dans les logs)
kubectl get pods -n ci-cd
kubectl logs <NOM_DU_POD_JENKINS> -n ci-cd

# Validation : ouvrir http://<IP_WORKER>:30000
```

1.6 Connexion Gitea ↔ Jenkins — Webhook CI/CD

L'automatisation CI/CD repose sur un mécanisme de webhook : quand du code est poussé sur Gitea, celui-ci envoie une requête HTTP à Jenkins qui déclenche automatiquement le pipeline. Plus de polling — le déclenchement est instantané.

Configuration côté Jenkins (le récepteur)

- Se connecter à Jenkins → Administrer Jenkins → Plugins → vérifier que le plugin « Gitea » est installé
- Créer un nouvel élément de type Pipeline
- Dans la configuration, cocher : « Build when a change is pushed to Gitea »

Configuration côté Gitea (l'émetteur)

- Se connecter à Gitea → ouvrir le dépôt → Paramètres → Webhooks
- Cliquer sur Ajouter un Webhook → Gitea
- URL cible : http://<IP_JENKINS>:30000/gitea-webhook/
- Type de contenu : application/json → Enregistrer
- Tester : bouton « Test Push » → icône verte = connexion établie

Pipeline de test — Jenkinsfile minimal

```
// Jenkinsfile – test de connexion Jenkins ⇔ Gitea
pipeline {
    agent any
    stages {
        stage('Test') {
            steps {
                echo 'Pipeline CI/CD opérationnel !'
            }
        }
    }
}
```

1.7 Conclusion — Résultats TP1

L'ensemble des objectifs du TP1 Kubernetes-CICD a été validé :

- Architecture Infrastructure — cluster Kubernetes stable avec 1 Master et 2 Workers sur Proxmox
- Cluster Kubernetes fonctionnel — kubectl get nodes affiche les 3 nœuds en état Ready (v1.30.14)
- Gitea déployé et configuré dans le namespace ci-cd (pod Running, interface web accessible)
- Jenkins déployé avec RAM 2 Gi, débloqué et prêt (Dashboard visible via port-forward)
- Accessibilité réseau — port-forward et tunnel SSH permettent d'accéder aux interfaces depuis un poste distant

- Webhook configuré — un push Git sur Gitea déclenche automatiquement un build Jenkins

> ☑ **OBJECTIFS VALIDÉS**
> L'« usine logicielle » est opérationnelle : Gitea héberge le code, Jenkins orchestre les builds, le webhook garantit le déclenchement automatique à chaque git push.

2. TERRAFORM — PROVISIONNEMENT IaC

Après l'orchestration Kubernetes, on s'attaque à l'Infrastructure as Code (IaC) avec Terraform : automatiser entièrement la création de VMs et de conteneurs sur Proxmox depuis un fichier texte déclaratif.

2.1 Concept — Infrastructure as Code avec Terraform

Au lieu de cliquer manuellement dans l'interface Proxmox pour créer une VM, on écrit un fichier .tf qui décrit l'état désiré. Terraform lit ce fichier et donne les ordres à Proxmox via son API.

Trois mécanismes clés :

- Provider — « traducteur » Proxmox. Terraform ne connaît pas Proxmox nativement ; le provider Telmate/Bpg sait dialoguer avec son API REST
- Plan — Terraform compare l'état désiré (fichiers .tf) avec l'état réel sur le serveur et génère une liste d'actions (créer, modifier, détruire)
- State (terraform.tfstate) — fichier mémoire de Terraform : il y consigne tout ce qu'il a créé afin de pouvoir le retrouver et le détruire en une seule commande (terraform destroy)

2.2 Initialisation du Projet Terraform

Créer un dossier tp2-terraform sur le Master et y placer le fichier main.tf avec la déclaration du provider Proxmox :

```
# main.tf — Déclaration du provider
terraform {
  required_providers {
    proxmox = {
      source  = "telmate/proxmox"
      version = "2.9.11"
    }
  }
}

provider "proxmox" {
  pm_api_url          = "https://<IP_PROXMOX>:8006/api2/json"
  pm_api_token_id     = "votre_token_id"
```

```
  pm_api_token_secret = "votre_token_secret"
  pm_tls_insecure     = true
}
```

```
# Télécharger le provider et initialiser l'environnement
terraform init
```

2.3 Déclarer la Ressource VM

Ajouter la ressource VM à la suite du fichier main.tf. Le mot-clé resource crée un objet Proxmox ; clone fait référence à un template existant.

```
# main.tf — Ressource VM Linux
resource "proxmox_vm_qemu" "vm_linux" {
  name        = "vm-linux-Styve"
  target_node = "ERIS"         # Nom du serveur Proxmox
  clone       = 411            # ID du template source
  vmid        = 30002

  cores  = 2
  memory = 2048

  network {
    model  = "virtio"
    bridge = "vmbr0"
  }

  disk {
    size  = "20G"
    type  = "scsi"
  }
}
```

2.4 Déploiement — terraform apply

Trois commandes pour passer de la déclaration à l'infrastructure réelle :

- terraform plan — affiche ce que Terraform va faire sans rien modifier ("1 to add")
- terraform apply — exécute réellement la création sur Proxmox
- terraform destroy — supprime toutes les ressources gérées par ce projet

```
# Vérifier la syntaxe et prévisualiser les actions
terraform plan

# Créer l'infrastructure (confirmer avec "yes")
terraform apply

# Supprimer l'infrastructure
```

```
terraform destroy
```

Prérequis pour que le déploiement fonctionne :

- Fichier main.tf sans erreur de syntaxe (terraform validate)
- API Token Proxmox avec les droits de création de VM
- Template source (clone) présent dans Proxmox avec l'ID référencé

BONNES PRATIQUES IaC
Ne jamais versionner le fichier terraform.tfstate ni les fichiers .tfvars contenant les secrets. Ajouter ces entrées dans .gitignore : .terraform/, *.tfstate, *.tfstate.backup, *.tfvars, *.pem